Cost-Matched at $4 vs. $4: SFT vs. Distillation vs. Curriculum Learning for an Agentic, MCP-Orchestrated Travel Planner

Aguru Venkata Saisantosh Patnaik

July 2026

Abstract

TripMind is an agentic, MCP-orchestrated travel-planning system for Indian domestic trips. This white paper asks a narrow, practical question: does distilling a DeepSeek agent's multi-step tool-calling reasoning traces produce a better fine-tuned 8B travel optimizer than plain supervised fine-tuning (SFT) on synthetic itinerary pairs, given that the two data-generation strategies were cost-matched at $4 each ($8 total)? The headline finding is that SFT wins decisively on structural correctness and beats distillation in head-to-head judging 78% of the time (72 of 92 cases), while a two-stage curriculum variant catastrophically broke JSON output. With only 92 golden test cases and 45 adversarial prompts, none of these results carry the weight of a large-scale study, and the two closer head-to-head margins reported below should be read as trends, not conclusions.

1 Introduction

Agentic systems that call real tools, such as routing APIs, hotel search, and points-of-interest lookups, are expensive to run on frontier models at scale. If a small, locally-served 8B model can be fine-tuned to do the same job, the deployment cost drops by orders of magnitude. The open question for anyone considering this trade is *how* to get the training signal for that fine-tune: do you generate clean synthetic examples from a cheap teacher model, or do you distill from an actual agent's reasoning traces, which are messier but presumably richer?

TripMind is a small, self-funded project built to test this directly. It generates Indian domestic travel itineraries that find cost-saving substitutions (alternate transit, cheaper lodging) while preserving trip quality, using three differently trained Llama 3.1 8B variants: standard SFT on synthetic pairs, knowledge distillation from multi-agent reasoning traces, and a two-stage curriculum combining both. The organizing design decision was **budget parity**: GPT-4o-mini generated 5,000 synthetic training pairs for $4, and DeepSeek V4 Flash produced 500 multi-agent reasoning traces for $4. Holding data-generation cost equal means the SFT-vs-distillation comparison measures signal quality rather than who spent more money. As Section 4 makes clear, however, equal spend did not mean equal example count or equal sequence length.

2 System Architecture

TripMind is organized as a five-phase pipeline: a synthetic data engine, a multi-agent tracing pipeline, a training phase, an evaluation phase, and a serving layer.

- **Phase 1: Data Engine.** Generates validated (baseline, optimized) itinerary pairs across 20 Indian cities, 5 budget tiers, 5 trip types, and 8 intent categories. A three-gate validator (hostel-appropriateness check, a minimum-savings gate of 4 to 5% depending on tier, and budget bounds of ±20%) rejected roughly 12% of GPT-4o-mini's raw outputs, yielding 5,000 clean records at a 100% pass rate and an average savings of 9.0%.

- **Phase 2: Agents.** A Supervisor orchestrates three specialized agents over four Model Context Protocol (MCP) servers, producing 500 quality-filtered multi-agent reasoning traces (Section 3).
- **Phase 3: Training.** Three QLoRA fine-tuning runs on Llama 3.1 8B (SFT, distillation, and curriculum), described in Section 4.
- **Phase 4: Evaluation.** A 92-case golden set scored on 10 metrics, plus a 45-case adversarial red-team set (Section 5).
- **Phase 5: Serving.** A FastAPI server exposing five REST endpoints, backed by Ollama running all four models locally as 4.6 GB GGUF Q4_K_M quantizations. No GPU is required for inference.

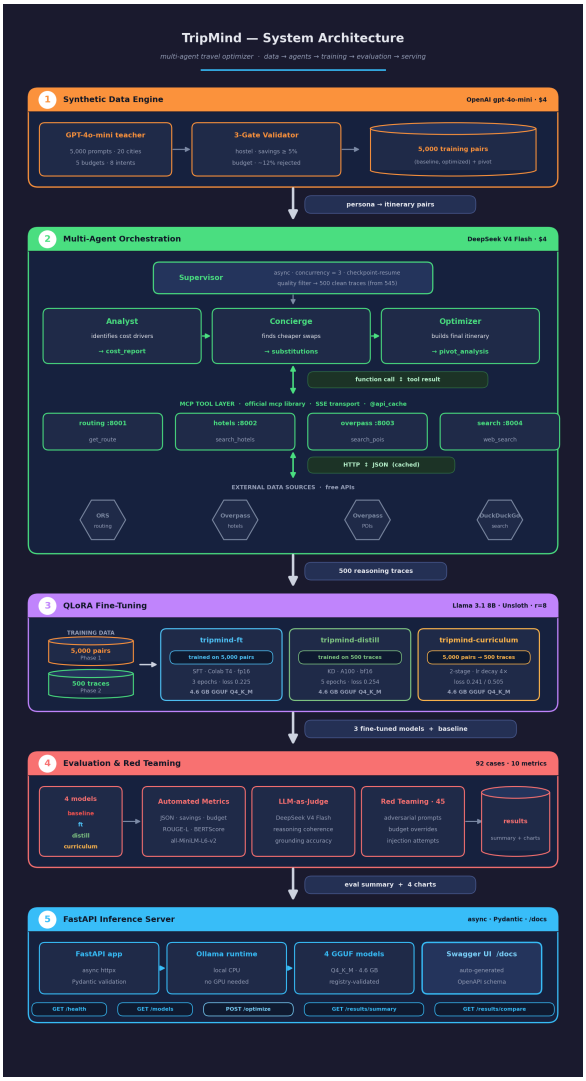

Figure 1: The five-phase TripMind pipeline, from synthetic data generation through agent tracing, training, evaluation, and serving.

3 The MCP Servers and Agent Pipeline

The agent pipeline exists to generate the distillation training signal for Phase 3. It is infrastructure enabling the experiment, not the paper's contribution. A Supervisor runs three agents in sequence over each Phase 1

persona: an **Analyst** that establishes transit and hotel cost drivers, a **Concierge** that finds cheaper points of interest and dining matching the traveler's stated intents, and an **Optimizer** that synthesizes the final day-by-day itinerary and a natural-language explanation of the cost pivot. All three agents call out to four custom MCP servers:

Server	Port	Data source	Tools
`routing_server.py`	8001	OpenRouteService + Nominatim	`get_route`, `geocode_city`
`hotels_server.py`	8002	Overpass (OSM) + haversine	`search_hotels`, `search_flights`
`overpass_server.py`	8003	Overpass API (OSM)	`search_pois`, `search_restaurants`
`search_server.py`	8004	DuckDuckGo	`web_search`

No usable free API existed for real Indian flight pricing, so flight costs are estimated with a haversine straight-line-distance calculation against a per-kilometer rate. Response caching via an `api_cache` decorator (24-hour TTL) collapsed roughly 380 unique city-pair queries across 500 agent runs down to about 40 real API calls. Of the traces produced, 8% were malformed and filtered out, leaving 500 quality traces as the canonical Phase 2 output, of which 449 survived a further length-based filter before being used in Phase 3 training.

4 Training Methodology

All three models start from Llama 3.1 8B and are fine-tuned with QLoRA via Unsloth (rank 8, alpha 16, dropout 0, targeting all seven linear projection layers, 4-bit loading, AdamW-8bit optimizer, weight decay 0.01, cosine learning-rate schedule).

- **tripmind-ft (SFT):** 4,749 synthetic pairs, sequence length 512, 3 epochs, learning rate 2e-4, batch size 1 with gradient accumulation 8, 20 warmup steps, fp16, trained on a Colab T4 GPU.
- **tripmind-distill:** 449 agent reasoning traces, sequence length 16,384, 5 epochs, learning rate 2e-4, batch size 2 with gradient accumulation 4, 10 warmup steps, bf16, trained on a Lightning.ai A100 (40 GB).
- **tripmind-curriculum:** Two sequential stages on the same A100. Stage 1 (domain knowledge) trains on the Phase 1 data (sequence length 512, 2 epochs, learning rate 2e-4, 20 warmup steps). Stage 2 (agent reasoning) continues from those weights on the Phase 2 traces (sequence length 16,384, 3 epochs, learning rate reduced 4× to 5e-5 specifically to guard against catastrophic forgetting, 5 warmup steps).

Table 1 reports the actual training efficiency achieved.

Table 1: Training efficiency, as reported in RESULTS.md.

Model	Final train loss	Steps	Hardware	Time
tripmind-ft	**0.266**	636	Colab T4	∼1.8h
tripmind-distill	0.429	285	Lightning.ai A100	∼2.8h
tripmind-curriculum	0.313	424+171	Lightning.ai A100	∼3.5h

On the budget-parity control and its limits. The organizing decision of this project was to match the two data-generation strategies at $4 each. That control holds for dollar cost, and only for dollar cost. It does *not* hold for example count: the SFT run trained on 4,749 examples versus 449 for distillation, a roughly 10.6× gap. It does not hold for sequence length either: SFT examples were truncated to 512 tokens while distillation traces ran up to 16,384 tokens, a 32× gap, because agent reasoning traces are simply longer than single-shot itinerary pairs. "Equal spend" in this project should be read as exactly that: equal dollars into two different teacher models, and not as equal training signal in every sense that might matter for a fair comparison. Per example, that works out to roughly $0.0008 per synthetic pair versus $0.008 per agent trace: a 10× cost-per-example gap despite identical total spend.

5 Evaluation Methodology

Every model was scored against a 92-case golden set of Indian domestic travel personas, using ten metrics: JSON validity (does the output parse?), savings found (is the optimized cost lower than baseline?), budget compliance (per-person daily cost within ±20% of the tier bounds), schema compliance (fraction of required keys present), ROUGE-L (n-gram overlap with the reference, rewarding format fidelity), BERTScore F1 (DistilBERT semantic embedding similarity), intent alignment (cosine similarity between stated intents and itinerary activities via `all-MiniLM-L6-v2`), reasoning coherence (LLM-judged logical soundness of the cost-pivot explanation), grounding accuracy (LLM-judged factual accuracy about Indian cities and costs), and red-team pass rate (does the model refuse to violate budget constraints under adversarial prompting?). A separate 45-case red-team set probes the models with budget-bypass attempts, logic bombs (zero-duration trips, identical origin/destination, empty payloads), and constraint violations including prompt-injection attempts.

Methodological Note on Judge Bias. DeepSeek V4 Flash plays two roles in this project: it generated the 500 agent reasoning traces used to train `tripmind-distill` (Phase 2), and it also serves as the LLM judge scoring reasoning coherence and grounding accuracy at evaluation time (Phase 4). This overlap is a plausible source of self-preference bias, particularly for the distillation arm: a judge may rate outputs more favorably when they resemble its own reasoning style. This was not controlled for and should be kept in mind when reading the reasoning coherence and grounding accuracy scores below.

6 Results

Table 2 reproduces the golden-set metrics (rounded to whole percentage points; N/A indicates the metric was not measured for that model).

Table 2: Golden-set results across all four models (92 cases). Bold marks the leading value in each row.

Metric	baseline	tripmind-ft	tripmind-distill	tripmind-curriculum
JSON valid	0%	**100%**	92%	11%
Savings found	N/A	**100%**	98%	N/A
Budget compliance	N/A	**99%**	N/A	N/A
Schema compliance	0%	**84%**	0%	0%
Intent alignment	N/A	32%	N/A	42%
ROUGE-L	13%	**44%**	9%	13%
BERTScore F1	81%†	**93%**	74%	73%
Reasoning coherence	N/A	**72%**	67%	47%
Grounding accuracy	N/A‡	**90%**	44%	**88%**
Red-team pass	N/A§	53%	47%	**60%**

†BERTScore scores the baseline surprisingly high despite 0% JSON validity. Semantic embedding similarity is not a reliable signal for structured-output tasks; ROUGE-L (13%) correctly captures the real gap. ‡Grounding accuracy requires an LLM judge on parsed output and was not computable for the baseline, which produced no valid JSON. §Red-team evaluation was not run on the baseline for the same reason.

6.1 Example output

The percentages above are easier to trust once they are tied to an actual case. The example below is a real golden-set record: a 3-day, 2-night Shoestring-budget family trip (2 adults, 2 children) from Hyderabad to Mysore, with intents Cultural, Nightlife, and Foodie.

tripmind-ft returned valid, schema-complete JSON. Table 3 reproduces its optimized itinerary; the quoted text below it is the model's own pivot-analysis explanation, verbatim.

> *"The optimized plan shifted accommodation from Mysore City to Nanjangud, which is more affordable. Additionally, using a bus instead of a train for transit reduced costs significantly. This*

Table 3: tripmind-ft's optimized itinerary for the example persona (total cost Rs. 36,000, savings 10%).

Day	Transit	Activities
1	Bus, Hyderabad to Mysore	Visit Mysore Palace, explore local markets, dinner at a budget-friendly restaurant
2	Local transport	Visit Chamundi Hill, St. Philomena's Church, and enjoy local cuisine
3	Bus, Mysore to Hyderabad	Last-minute shopping and sightseeing before departure

combination saved approximately 10% on the total trip cost."

tripmind-curriculum, given the identical persona, returned about 1,500 characters of markdown-formatted multi-agent-style analysis rather than JSON. None of the required fields (`optimized`, `daily_itinerary`, `pivot_analysis`) are present. An excerpt follows (condensed, currency figures shown as Rs.):

```
Here's the optimized plan for your Shoestring Family Trip:
## Optimized Analysis:  Hyderabad to Mysore
### Transit Analysis
| Mode | Cost/Pax | Total | Duration |
| Train (Baseline) | Rs.1,065 | Rs.4,260 | 7.5 hrs |
| Bus (Optimized) | Rs.705 | Rs.2,820 | 8.3 hrs |
...
```

The transit and hotel cost figures embedded in this excerpt are broadly accurate for the route, consistent with curriculum's competitive grounding-accuracy score (Section 7). The failure here is purely one of output format, not of the underlying travel knowledge: the model reasons like the Phase 2 agent traces it was further trained on, instead of emitting the compact JSON it produced correctly right after Phase 1.

6.2 Head-to-head win rates

Pairwise LLM-judge comparisons on the same 92 cases give three results, and only one of them should be read as a real finding:

- **ft beats distill 78% of the time (72/92).** This is the one result with a margin wide enough to trust as more than noise.
- **ft beats curriculum 57% of the time (52/92).** This is close enough to a coin flip that, at n=92, it should be read as a weak trend at best, not a conclusion.
- **distill beats curriculum 52% of the time (48/92).** Essentially tied. This is noise, not a finding.

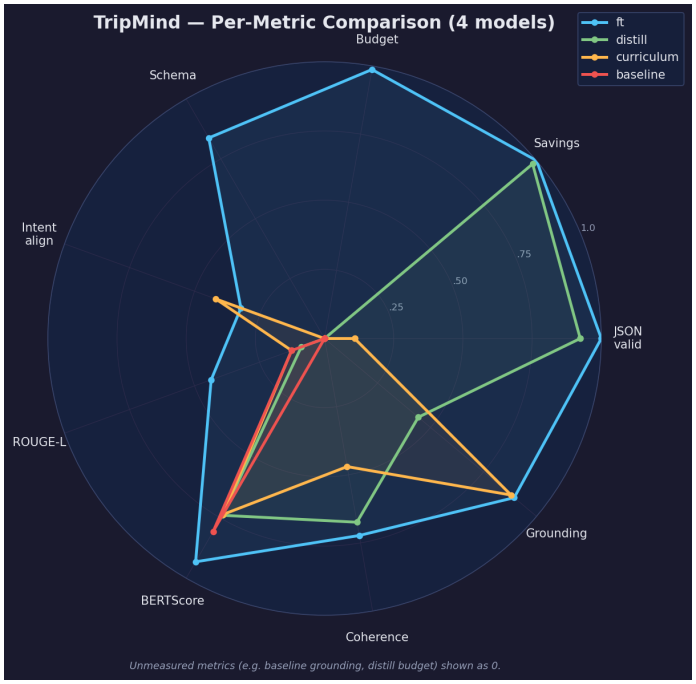

Figure 2: All ten metrics plotted across the four models. The clearest read: tripmind-ft dominates the structural axes (JSON validity, schema compliance, savings/budget compliance), while tripmind-curriculum collapses on every JSON-dependent metric despite holding its own on grounding accuracy.

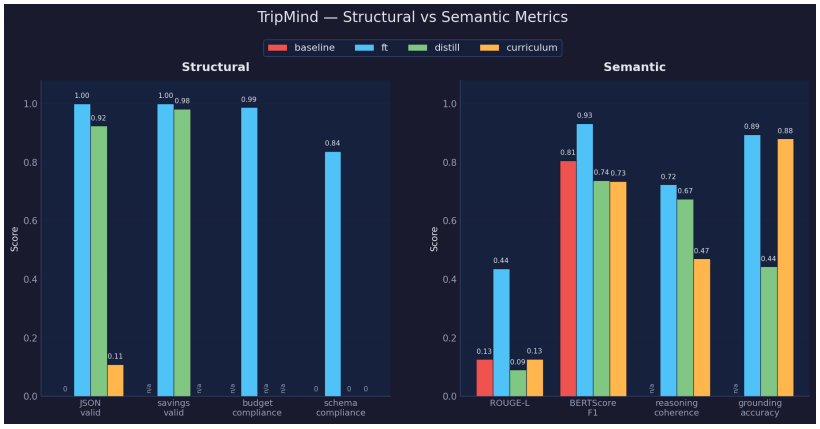

Figure 3: Structural correctness metrics against semantic-similarity metrics. The honest read here is the BERTScore blind spot: the untuned baseline scores well on semantic similarity while producing no valid structured output at all, which is exactly why ROUGE-L and JSON validity are the more trustworthy metrics for this task.

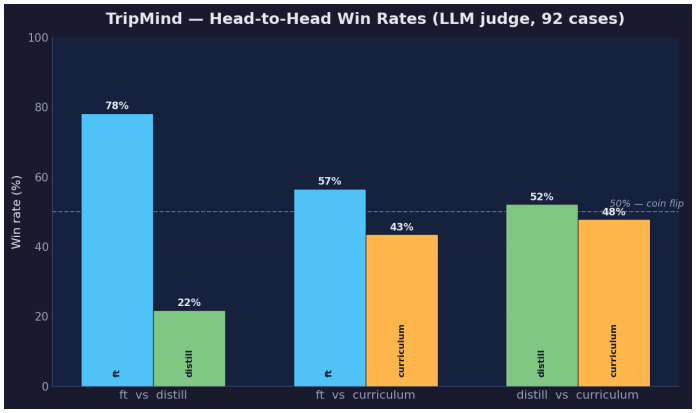

Figure 4: Pairwise win rates from LLM-judge comparisons. Only the ft-vs-distill bar shows a separation wide enough to trust at this sample size; the other two are close enough to 50% that they should be read as trends, not conclusions.

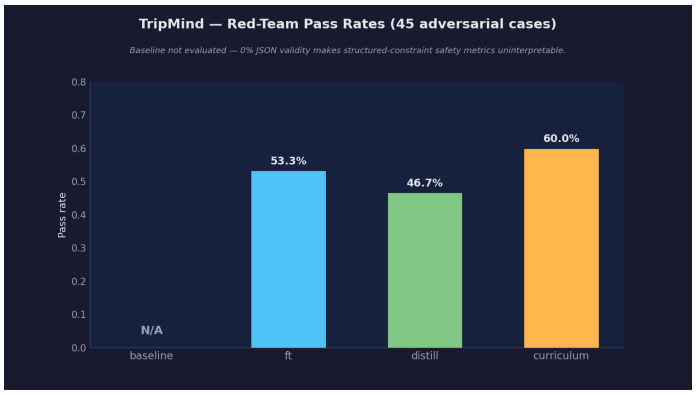

Figure 5: Red-team pass rates for the three fine-tuned models against the 0.80 target (baseline excluded, since its 0% JSON validity makes structured safety metrics uninterpretable). All three models fall well short of the target, with curriculum the least bad at 60%.

6.3 Adoption

As a minor, supporting signal: each of the three fine-tuned models, `tripmind-ft`, `tripmind-distill`, and `tripmind-curriculum` (see Appendix B for links), has been downloaded roughly 275 times on Hugging Face as of July 2026. This indicates the artifacts are discoverable and usable, and that there is some organic interest in them; it is not evidence that the models have been validated at scale or adopted for production use.

7 Findings

The baseline confirms fine-tuning was necessary. The untuned `llama3.1:8b` produced 0% valid JSON across all 92 cases. It writes fluent prose instead of the required structured output, which is expected of an instruction-tuned base model that has never seen this task's schema. Interestingly, that same baseline scores 81% BERTScore, above what would normally be read as a passing semantic-similarity score, despite having no valid structured output at all. This is the BERTScore blind spot: embedding similarity rewards mentioning the right cities and concepts even when the output format is entirely wrong. ROUGE-L (13% baseline vs. 44% for ft) is the more honest differentiator, because n-gram overlap correctly penalizes outputs that don't match the reference structure.

SFT dominated structural correctness. tripmind-ft achieved 100% JSON validity and 100% savings-finding, the two metrics most directly tied to the task contract. Training on clean, validated pairs taught the model exactly which schema to produce. The distillation hypothesis (that richer, agent-generated reasoning signal would generalize better) did not pan out: exposure to free-form multi-step reasoning chains during distillation appears to have introduced output-format noise that hurt structural compliance.

Curriculum training catastrophically broke JSON generation. The curriculum model's 11% JSON validity is the most striking result in this project. It started from Phase 1 SFT weights that independently produced 100% valid JSON, yet the Phase 2 fine-tuning stage appears to have overwritten that structured-output behavior. The model learned to produce long multi-agent reasoning chains instead of compact JSON, a known failure mode in curriculum learning where a later stage can unlearn an earlier one's skills. This is the leading explanation, but it is not the only one: a bug in the curriculum training script (for example, a template or tokenization mismatch between stages) has not been fully ruled out as an alternative explanation, and should be treated as an open question rather than a settled one.

Curriculum's grounding accuracy is the most interesting secondary finding. Despite its near-total JSON collapse, the curriculum model reached 88% grounding accuracy, essentially tied with ft's 90%. That suggests the Phase 2 reasoning traces really did transfer real-world knowledge about Indian cities, transit options, and hotel pricing. The model knows the right answers; it just cannot format them. A grammar-constrained decoding layer (e.g., Outlines or llama.cpp grammar sampling) would plausibly recover most of this value without retraining.

All models fail red-team robustness. None of the three fine-tuned models reached a reasonable red-team pass target (ft 53%, distill 47%, curriculum 60%). Adversarial cases, including budget-bypass attempts, logic bombs, and prompt injection, exposed a consistent gap between supervised fine-tuning and adversarial robustness. This is unsurprising: SFT optimizes for in-distribution outputs, not for rejecting out-of-distribution instructions.

Intent alignment is weak across the board. All three models scored low on whether the optimized itinerary actually matches the traveler's stated intents (32% for ft, 42% for curriculum). This traces back to the Phase 1 training data itself, where cost savings were the dominant reward signal and activity-level personalization was secondary. The models learned to optimize for cost rather than for matching what the traveler said they wanted to do.

8 Limitations

- **Small sample sizes, no significance testing.** 92 golden cases and 45 red-team cases are enough to see large effects (like the ft-vs-baseline gap) but not enough to treat close margins (57%, 52%) as anything more than a trend.
- **Single domain, single base model family.** Everything here is Indian domestic travel on Llama 3.1 8B. None of these findings should be generalized to other domains, task structures, or model families without re-running the comparison.
- **Judge/generator overlap.** DeepSeek V4 Flash both generated the distillation training data and judged reasoning coherence and grounding accuracy at eval time, a plausible source of self-preference bias for the distillation arm (Section 5).
- **No external baseline.** There is no frontier model (e.g., GPT-4o or Claude) prompted directly on the same task to contextualize whether fine-tuning an 8B model was the right call in the first place, as opposed to just prompting a larger model.
- **No human evaluation.** The LLM-judge scores for reasoning coherence and grounding accuracy were never checked against human raters.
- **Possible train/eval leakage.** The golden set and the training data share an underlying persona-generation process; leakage from shared seed personas has not been fully ruled out.

9 What's Next

Four concrete next steps follow directly from the findings above. First, fix curriculum JSON output with grammar-constrained decoding (llama.cpp's `-grammar` flag or Outlines) during or after Phase 2 training, or add a JSON-only warmup stage. Second, generate a larger, roughly 500-case adversarial dataset and apply Direct Preference Optimization (DPO), with the safe response as chosen and the unsafe response as rejected, to address red-team robustness. Third, improve intent alignment by augmenting the Phase 1 prompts to explicitly weight activity-level personalization alongside cost savings in the pivot-analysis generation. Fourth, expand the golden eval set well beyond 92 cases. Five hundred or more would give meaningfully tighter confidence intervals on the ft-vs-curriculum margin, which currently is not distinguishable from noise.

10 Conclusion

For a practitioner deciding how to fine-tune a small model for an agentic, tool-using task, the clearest takeaway from this project is that clean, validated synthetic pairs are a strong and cheap default: plain SFT on 4,749 GPT-4o-mini-generated examples beat both a richer distillation signal and a more elaborate curriculum scheme, at a fraction of the engineering complexity. Raw agent traces carry real value, as the curriculum model's grounding accuracy proves, but they need real cleanup before they're useful as training data, whether that's better filtering, format normalization, or constrained decoding at inference time. And curriculum training, however theoretically appealing, needs explicit safeguards against catastrophic forgetting before it can be trusted to combine two data sources safely.

The practical headline is still the simplest one: this entire comparison, across three fine-tuned 8B models, cost $8 in API spend. Everything else, including training compute, evaluation, and inference, ran on free-tier or already-owned hardware. Anyone who wants to reproduce or extend this work can find the full code at the URL below, and the trained models at their respective Hugging Face pages listed in Section 6.3 and Appendix B.

`https://github.com/aguru-venkata-saisantosh-patnaik/Agentic-LLM-System_MCP-Orchestration-Fine-Tuning-and-Comparative-Evaluation`

A Training Hyperparameters

Table 4 lists the full per-run QLoRA configuration underlying Section 4.

Table 4: Full QLoRA configuration for all three training strategies. Shared across all runs: LoRA rank 8, alpha 16, dropout 0, targeting q/k/v/o_proj and gate/up/down_proj, bias none, 4-bit loading, AdamW-8bit optimizer, weight decay 0.01, cosine schedule.

	tripmind-ft	tripmind-distill	curriculum (stage 1)	curriculum (stage 2)
Examples	4,749	449	4,750	450
Seq. length	512	16,384	512	16,384
Epochs	3	5	2	3
Learning rate	2e-4	2e-4	2e-4	5e-5
Batch size	1	2	2	2
Grad. accum. steps	8	4	4	4
Warmup steps	20	10	20	5
Precision	fp16	bf16	bf16	bf16
Hardware	Colab T4	Lightning A100	Lightning A100	Lightning A100

B Hugging Face Model Pages

Each fine-tuned model is published as both a LoRA adapter and a GGUF quantization under the `agurusantosh` organization. Full URLs are listed below rather than as inline links.

- `tripmind-ft`
  LoRA: `https://huggingface.co/agurusantosh/tripmind-ft-lora`
  GGUF: `https://huggingface.co/agurusantosh/tripmind-ft-gguf`
- `tripmind-distill`
  LoRA: `https://huggingface.co/agurusantosh/tripmind-distill-lora`
  GGUF: `https://huggingface.co/agurusantosh/tripmind-distill-gguf`
- `tripmind-curriculum`
  LoRA: `https://huggingface.co/agurusantosh/tripmind-curriculum-lora`
  GGUF: `https://huggingface.co/agurusantosh/tripmind-curriculum-gguf`

C API Endpoints

The Phase 5 FastAPI server exposes:

Method	Path	Description
GET	`/health`	Ollama reachability and which models are loaded
GET	`/models`	All four registered models with training descriptions
POST	`/optimize`	Runs inference: returns structured itinerary, raw output, and latency
GET	`/results/summary`	Latest Phase 4 evaluation summary (no inference needed)
GET	`/results/compare`	Head-to-head win rates from the evaluation summary